Architecture.md v1.1

Document: Software Architecture Specification (SAS)

Project Name: AI OS

Version: 1.1

Status: Official Architecture Blueprint

Chief Architect: ChatGPT

Project Owner: Ayush

Chapter 1 — System Overview

1.1 Purpose

AI OS is a modular, intelligent operating system designed for a single user. It is not intended to function as a traditional chatbot. Instead, it serves as a centralized AI platform that coordinates multiple intelligent agents to assist with decision-making, productivity, business management, analytics, research, automation, and future capabilities.

The architecture is designed to support long-term expansion without requiring structural redesign.

1.2 Objectives

The system shall:

- Provide natural language interaction through two primary AI personalities.
- Coordinate specialized worker agents through a central orchestration layer.
- Maintain a single, shared memory system.
- Support modular expansion through a plugin-based agent architecture.
- Keep all user data centralized and consistent.
- Allow replacement of AI models without changing system architecture.
- Separate conversation, orchestration, business logic, and persistence into independent layers.

1.3 Core Principles

Modularity

Every subsystem shall operate independently and expose clearly defined interfaces.

No component shall depend directly on the internal implementation of another component.

Scalability

The architecture must support:

- Additional worker agents
- New AI models
- New databases
- New interfaces
- New automation systems

without requiring modification of existing modules.

Separation of Responsibilities

Every component has exactly one responsibility.

Examples:

- Router decides destination.
- Jarvis performs strategic reasoning.
- Friday performs operational reasoning.
- Agent Manager coordinates execution.
- Memory Service stores information.
- Worker Agents perform specialized tasks.

Responsibilities must never overlap unnecessarily.

Shared Intelligence

Jarvis and Friday shall operate from one shared knowledge base.

There shall never be multiple independent memory systems for different personalities.

User Authority

The user always has final authority.

The system may recommend, automate, or assist, but it shall never execute sensitive actions without explicit permission.

Replaceability

Every major module must be replaceable.

Examples include:

- AI Model
- Database
- Memory implementation
- Analytics module
- Trading module
- Voice engine

Replacing a module must not require redesigning the architecture.

1.4 System Layers

AI OS is divided into seven logical layers.

Layer 1 — User Layer

Responsible for:

- Text input
- Voice input
- Dashboard interaction

This is the only layer directly accessed by the user.

Layer 2 — Routing Layer

Responsible for:

- Receiving requests
- Classifying intent
- Selecting Jarvis or Friday

No business logic exists in this layer.

Layer 3 — Personality Layer

Contains:

- Jarvis
- Friday

Responsibilities:

- Conversation
- Planning
- Delegation
- Result explanation

Personalities never execute business logic directly.

Layer 4 — Orchestration Layer

Contains:

- Agent Manager

Responsibilities:

- Task routing
- Workflow sequencing
- Context distribution
- Response aggregation
- Duplicate prevention

No worker agent communicates outside this layer.

Layer 5 — Worker Layer

Contains specialized agents such as:

- Planner
- Analytics
- Business
- Trading
- Content
- Finance
- Browser

- Vision
- Research
- Calendar
- Email
- Automation
- PC Control

Each worker agent performs exactly one specialized responsibility.

Layer 6 — Core Services Layer

Contains:

- Memory Service
- Configuration Service
- Logging Service
- Permission Service
- Event Service

These services support every other layer while remaining independent of business logic.

Layer 7 — Persistence Layer

Responsible for:

- Databases
- File storage
- Vector memory
- Configuration files
- Logs

Only Core Services communicate with this layer.

Worker agents never access persistence directly.

1.5 High-Level System Flow

The standard request lifecycle is defined as follows:

```
User
  │
  ▼
Router
```

```
   |
   ▼
Jarvis / Friday
   |
   ▼
Agent Manager
   |
   ▼
Worker Agent(s)
   |
   ▼
Memory Service
   |
   ▼
Agent Manager
   |
   ▼
Jarvis / Friday
   |
   ▼
User
```

Every request shall follow this pipeline unless explicitly defined otherwise in later architectural chapters.

1.6 Architectural Boundaries

The following interactions are prohibited:

- User → Worker Agent
- Worker Agent → Worker Agent
- Worker Agent → Database
- Worker Agent → User
- Jarvis → Database
- Friday → Database

All communication shall occur through the designated architectural layers.

1.7 Success Criteria

The architecture shall be considered successful when it satisfies the following requirements:

- Components remain independently replaceable.
- New agents can be added without redesign.

- User memory remains centralized.
- AI personalities remain synchronized.
- Business logic remains isolated from conversation logic.
- Every module can be developed, tested, and maintained independently.
- The system remains understandable as it grows.

End of Chapter 1

Next Chapter: Chapter 2 — Design Philosophy & Architectural Principles